DGL∞
CLASSICAL GRAPH REPRESENTATION: TYPES, PROPERTIES & TASKS.

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.

https://github.com/udlbook/

What is a graph? [UDL-13]

Graphs coded into matrices

Graphs are powerful tools in modeling interactions/relationships between different elements.
By modeling such dependencies/interactions in a reliable way, one can derive rules, predictions or new knowledge about the system in hand.

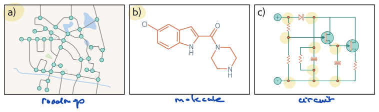


Figure 1 Real-world graphs. Some objects such as a) road networks, b) molecules, and c) electrical circuits are naturally structured as graphs.

- Social networks are graphs where nodes are people and the edges represent friendships between them.
- The scientific literature can be viewed as a graph in which the nodes are papers and the edges represent citations.
- Wikipedia can be thought of as a graph in which the nodes are articles and the edges represent hyperlinks between articles.
- Computer programs can be represented as graphs in which the nodes are syntax tokens (variables at different points in the program flow) and the edges represent computations involving these variables.
- Geometric point clouds can be represented as graphs. Here, each point is a node and has edges that connect to other nearby points.
- Protein interactions in a cell can be expressed as graphs, where the nodes are the proteins, and there is an edge between two proteins if they interact.

More examples

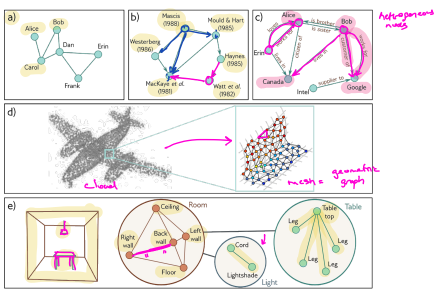


Figure 2 Types of graph. a) A social network is an undirected graph; the connections between people are symmetric. b) A citation network is a directed graph; one publication cites another, so the relationship is asymmetric. c) A knowledge graph is a directed heterogeneous multigraph. The nodes are heterogeneous in that they represent different object types (people, places, companies) and there may be multiple edges representing different relations between each node. d) A point set can be converted to a graph by forming edges between nearby points. Each node has an associated position in 3D space and this is termed a geometric graph (adapted from Hu et al. 2021). e) The scene on the left can be represented by a hierarchical graph. The topology of the room, table, and light are all represented by graphs and these graphs form nodes in a larger graph representing object adjacency (adapted from Fernández-Madrigal & González 2002).

Graph representation into a matrix

$G = (A, V, E)$ — adjacency matrix; set of nodes; e_{ij} connects node i to j

(A, X, E) — embedding / feature / annotation

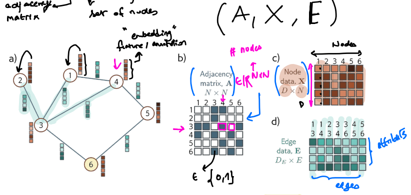


Figure 3 Graph representation. a) Example graph with six nodes and seven edges. Each node has an associated embedding of length five (brown vectors). Each edge has an associated embedding of length four (blue vectors). This graph can be represented by three matrices. b) The adjacency matrix is a binary matrix, where the element (m, n) is set to one if node m connects to node n. c) The node data matrix $\mathbf{X}$ contains the concatenated node embeddings. d) The edge data matrix $\mathbf{E}$ contains the edge embeddings.

Graph types

1. Directed vs undirected/ binary vs weighted

(0,1) edge weight

edge weight $w_{ij} \in \mathbb{R}$, $\forall i,j \in V$

directed e_{ij} ~~e_{ji}~~

undirected $w_{ij} = 0.5$

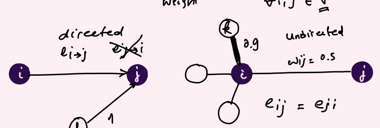
$e_{ij} = e_{ji}$

$e_{ik} = 0$

$e_{k \to j} = 1$

$w_{ik} = 0.9$

$w_{ij} = 0.5$



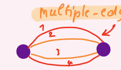
2. Simple graph

A graph is simple if it has no loops or multiple edges.

loop

self-connection [an edge that connects up the same node]

multiple-edge



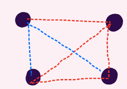
Two edges that connect the same end points (or more). e.g.: two routes connecting two cities.

3. Complete graph

A complete graph has each of its nodes connected to all other nodes.

→ every two nodes are adjacent.

N nodes in G; $\forall$ node i connects to $(N-1)$ nodes; N nodes including self-connection



Graph types

8. Hypergraphs

→ models high-order relationships among nodes. ≠ typical graphs ⇒ low-order

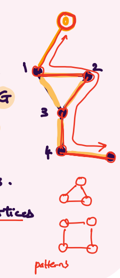
A hypergraph H is a generalization of a graph in which an edge can join any number of vertices.

→ It is a set of non-empty subsets (or hyperedges).

4. Subgraphs

examine the structure of graphs that lie within a given graph.

- A graph H is called a subgraph of G if every vertex and edge of H belong to G: $V(H) \subseteq V(G)$ & $E(H) \subseteq E(G)$
- A graph G is a subgraph of itself.
→ all other subgraphs are proper subgraphs.
- If H is a subgraph of G having the same vertices as G, then H is a spanning subgraph of G.



7. Heterogeneous graphs

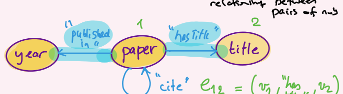
A heterogeneous graph $G = (V, \mathcal{E}, R, T)$ → $U(X, E)$ node feature matrix; edge feature matrix

relation type

nodes with node types $v_i \in V$

edges with relation types $(v_i, r, v_j) \in \mathcal{E}$ — triplet encoding the relationship between pairs of nodes

nodes and edges have features/attributes



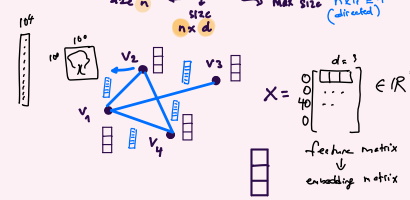
6. Annotated/decorated graphs

nodes / edges

A graph G is annotated or decorated if its nodes can hold attributes / features.

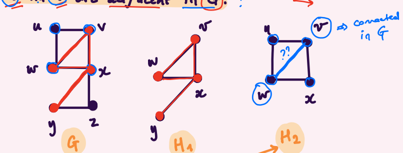
$G = ((V, X), E)$

size n; size $n \times d$; max size $n \times n = n^2$ (directed)



5. Induced subgraphs

- Given a non-empty set $S \neq \phi$ of nodes of G, the subgraph $G[S]$ of G induced by S has vertex set S where two vertices u and v are adjacent if and only if (iff) u and v are adjacent in G.



Graph types

BASIRA https://basira-lab.com/

YouTube Search "BASIRA Lab"

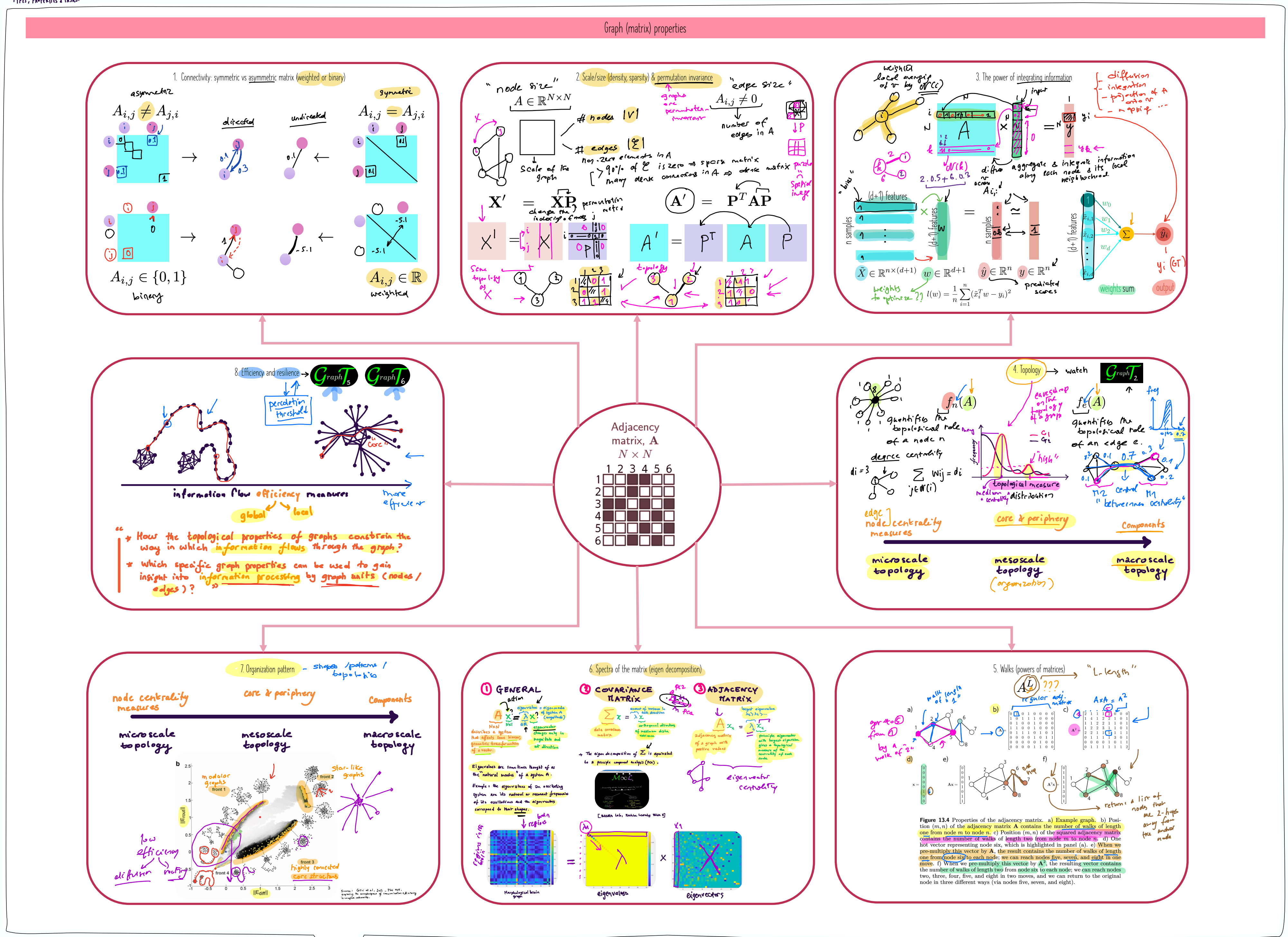
https://github.com/basiralab

DGL∞

CLASSICAL GRAPH REPRESENTATION: TYPES, PROPERTIES & TASKS.

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.

https://github.com/udlbook/



BASIRA

https://basira-lab.com/

YouTube

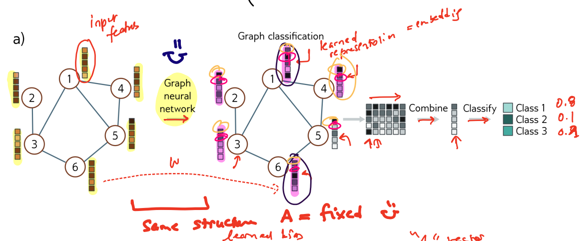
Search "BASIRA Lab"

https://github.com/basiralab

Learning tasks

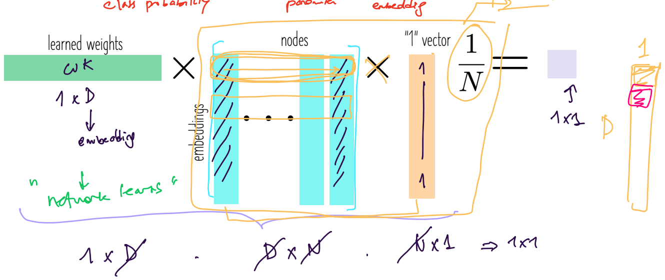
Graph classification

Classify a graph (e.g., molecule — poisonous or not)

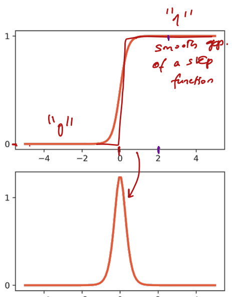


$$y = \mathrm{sig}\left[\beta_K + \omega_K \mathbf{H}_K \mathbf{1}/N\right]$$

predicted class probability · sigmoid · learned bias · learned params · learned embedding · "1" vector · average / mean pooling



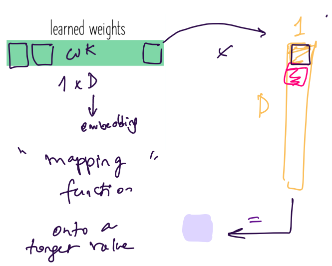
Logistic activation (sigmoid)



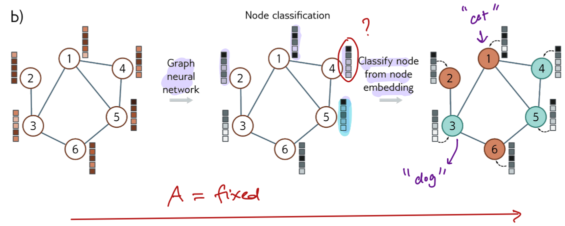
$$\sigma(x) = \frac{1}{1+e^{-x}}$$

"1" / "0" — "classification"

$$\frac{d}{dx}\sigma(x) = \sigma(x)\cdot(1-\sigma(x))$$

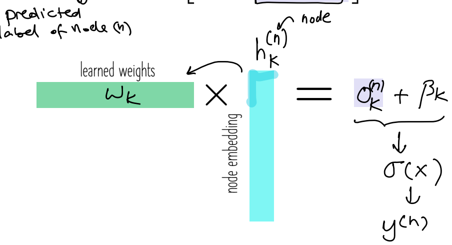


Node classification

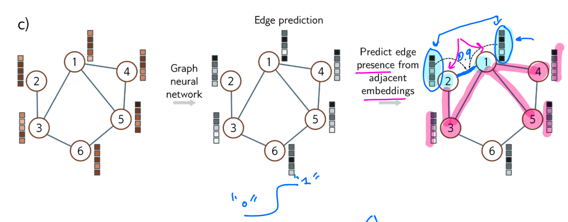


$$y^{(n)} = \mathrm{sig}\left[\beta_K + \omega_K \mathbf{h}_K^{(n)}\right]$$

predicted label of node (n)

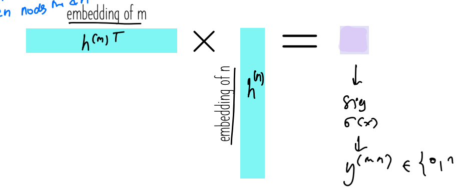


Edge prediction



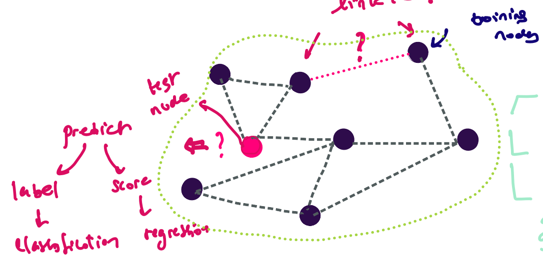
$$y^{(mn)} = \mathrm{sig}\left[\mathbf{h}^{(m)T}\mathbf{h}^{(n)}\right]$$

prob of link existence between nodes m and n



Generalized { 1) node-level, 2) edge-level, 3) graph level } prediction

link / edge prediction



label / classify
score / regress
generate another graph from it